

一元三次方程求解

有形如： $ax^3+bx^2+cx+d=0$ 这样的的一个一元三次方程。给出该方程中各项的系数(a, b, c, d 均为实数)，并约定该方程存在三个不同实根(根的范围在 -100 至 100 之间)，且根与根之差的绝对值 ≥ 1 。要求由小到大依次在同一行输出这三个实根(根与根之间留有空格)，并精确到小数点后2位。

提示：记方程 $f(x)=0$ ，若存在2个数 x_1 和 x_2 ，且 $x_1 < x_2$ ， $f(x_1)*f(x_2) < 0$ ，则在 (x_1, x_2) 之间一定有一个根。

输入：

1 -5 -4 20

输出：

-2.00 2.00 5.00

一元三次方程求解

对于一个方程而言，如果 $f(m) * f(n) < 0$ ，那么说明 $[m, n]$ 区间内至少有一个零点，改题已经明确说明根与根的绝对值只差 ≥ 1 ，那就说明对于 $f(m) * f(m+1) < 0$ ，说明 $[m, m+1]$ 内肯定有一个零点，否则没有零点。

所以我们可以依次遍历 $[-100, 100]$ 区间内的 $[m, m+1]$ ，如果满足条件就说明 $[m, m+1]$ 内肯定有一个解，同样的，如果 $f(m) == 0$ 就直接有解了。

当确定区间之后，我们可以用二分的方法缩小区间范围，如果 $f(\text{mid}) * f(\text{left}) < 0$ 说明在左区间内，反之就在右区间内，要求精确到小数点后两位，所以当 $\text{right} - \text{left} < 0.001$ 的时候就结束，随便输出 left 或者 right 。

注意数据范围都要用double

二分答案

上面方程求解这道题我们可以理解为二分查找区间内合理的值，也可以理解为二分答案即，已知答案在一个区间之内，我们通过二分的方法来缩小答案的范围达到找到答案的目的。这样查找相比较于顺序查找(逐个查找)，效率会高很多。

所以对于一些直接求答案比较麻烦的题，或者说完全没思路的题，当是我们又恰好知道答案在某个范围之内(最好是整数)，我们就可以通过枚举答案的方法来反向验证这个答案是否合理。既然答案是一个区间，区间肯定具有单调性的，那么我们可以用二分的办法来枚举答案，这就是二分答案，这种方法和数学方法有很大区别，要及时的转换思维。

STL 排序

头文件: `#include<algorithm>`

默认排序: `sort(a+1, a+n+1);`

排序方法: 把a[1]到a[n]从小到大排序。

自定义排序:

```
bool compare(int a, int b) //默认a<b
{
    return a>b; //从大到小排序
}
sort(a+1, a+n+1, compare);
```

丢瓶盖

陶陶是个贪玩的孩子，他在地上丢了A个瓶盖，为了简化问题，我们可以当作这A个瓶盖丢在一条直线上，现在他想从这些瓶盖里找出B个，使得距离最近的2个距离最大，他想知道，最大可以到多少呢？

输入格式：

第一行，两个整数，A, B。 ($B \leq A \leq 100000$)

第二行，A个整数，分别为这A个瓶盖坐标, $\leq 1e9$ 。

输出格式： 仅一个整数，为所求答案。

输入：

5 3

1 2 3 4 5

输入：

2

丢瓶盖

因为每个瓶盖的坐标未定，所以找不到任何算法可以直接得到最小距离的最大值，所以我们只能一个答案一个答案的去试，已知 $A[i] \leq 1e9$ ，所以一次枚举答案肯定会超时，已知坐标是整数，并且坐标有一个区间，区间是具有单调性的，所以我们可以用二分查找。

第一步：

题目中没说每个瓶盖的坐标是已经排好序的，所以先排序

```
#include<algorithm>
```

```
sort(a+1, a+n+1); //对a[1]到a[n]从小到大排序
```

第二步：

确定二分答案的左右端点，即最小可能距离，最大可能距离

```
int left=1; //左端点
```

```
int right=a[n]-a[1]; //右端点
```

丢瓶盖

第三步：

查找中间值，然后带入题目中判断是否满足条件。

假设最小值就是当前这个值，说明选取的瓶盖的距离一定是 $\geq$ 最小值，那么我们就从起点开始往后面找，只要距离 $\geq$ 最小值，就选取，然后统计最后一共可以选 k 个。

注意第一个点可以理解为肯定是必选的！

既然是假设，那么就可能有不满足条件的时候：

如果 $k < m$ ：说明满足条件的瓶盖少了，即最小值大了。

所以 $right = mid - 1$;

如果 $k = m$ ：说明刚好满足条件，最小值是正确的。

如果 $k > m$ ：说明满足条件的瓶盖多了，即最大值小了。

所以 $left = mid + 1$;

丢瓶盖

实际上前面的想法是错误的，因为我们要求的是要让他
的最小值最大。求最大的最小值，而不是直接求一个满足条
件的最小值。

If ($k < m$): 说明最小值大了，最多也没有办法选择 m 个。

If ($k == m$): 说明最小值是合理的，但是不一定是最大的，
可能还存在稍微大点的值合适合理的。比如

1 4 9 (3选3)

第一次 $mid=4$ ，不满足条件，第二次 $mid=2$ ，满足条
件，但是实际值应该是3

If ($k > m$): 说明最多可以选择的瓶盖比 m 多，但是并不代
表不可以选 k 个，所以这种情况也是合理的，但是也不
一定是最小值。

1 2 3 4 (4选3)

最小值只能是1，但是是1时可以选4个。

二分答案模板

```
while(left<=right)
{
    //求最小值的最大值。
    int mid=(left+right)/2;
    if(check(mid)) //满足条件
    {
        ans=mid; //记录当前答案
        left=mid+1; //查找有没有更大的值
        //如果有更大的值，就会自动更新ans
    }
    else
    {
        right=mid-1; //不合理，肯定不满足条件
    }
}
printf("%d", ans);
```

切绳子

有N条绳子，它们的长度分别为 L_i 。如果从它们中切割出K条长度相同的绳子，这K条绳子每条最长能有多长？答案保留到小数点后2位（不四舍五入）。

输入：

4 11

8.02

7.43

4.57

5.39

输出：

2.00

对于100%的数据 $0 < L_i \leq 100000.00$

$0 < n \leq 10000$ $0 < k \leq 10000$

切绳子

前面我们使用的二分答案的模板是针对整数而言的，如果当前值不合理，那么就从当前值的下一个（上一个）整数开始继续寻找，但是该题就不可以这样做了，因为他可能是小数，所以我们就不可以 $\text{Left}=\text{mid}+1$, $\text{right}=\text{mid}-1$ 了，这样很容易丢失正解，因为正解可能就在相邻两个整数之间。

```
while(right-left>0.0001)
{
    double mid=(left+right)/2;
    if(check(mid))
    {
        ans=mid;
        left=mid;
    }
    else
        right=mid;
}
```

因为保留两位小数，所以要让left和right的值精确到两位小数数值肯定是一样的，所以这里精度要高一点。

切绳子

但是在实际做题中我，我们会发现一个问题，就是经过多次计算后，由浮点数只是一个近似值，会有一定误差，在一定限度内，误差可以忽略不计，但是多次计算之后，误差会导致结果错误。比如1.0000在多次计算之后会变成0.99999，这样最后的结果就是错误的。

比如洛谷上面切绳子这题，如果最后输出的答案是ans的值或者left的值，答案会错一部分，但是使用right的值就是正确的，但是理论上这两者的值精确到 $1e-4$ 是正确的，实际上就算精确到 $1e-6$ 都还是会错。所以当我们需需要精确到较高的时候最好不使用浮点数。最好想办法转换成整数再计算。

浮点数

```
double a=1.0;
while(a!=2.0)
{
```

```
    a+=0.1;
    printf("%lf\n", a);
```

```
}
```

死循环

```
int a, b, c;
scanf("%d%d%d", &a, &b, &c);
int x=a*0.2+b*0.3+c*0.5;
```

如果输入60 90 80 有的编译器结果会是78

在判断一个数是否是质数的时候，最好用 $i*i \leq n$ 而不是 $i \leq \text{sqrt}(n)$ ，比如对4开方的时候会变为1.999999

切绳子

所以切绳子这道题最好的做法就是把所有绳子长度扩大10000倍（保证保留两位小数时，近似值一样），然后按照前面对整数二分答案的做法来做。但是最后要求是保留两位小数，但是不四舍五入，所以当得到ans的值的时候，还要想办法变回原来的值，并且保留两位小数。

比如：ans=12358

正确结果应该是1.23，如果直接/10000，再保留整数结果是1.24.

洛谷 P1577 切绳子

切 繩 子